FCM II Graded Homework 4

Charles Ohanian

April 14, 2025

1 Executive Summary

This report investigates the accuracy and efficiency of six composite quadrature methods - Midpoint, Trapezoidal, Simpson's First, Left Rectangle, Open Newton-Cotes (two-point), and Gauss-Legendre (two-point) - for approximating definite integrals. Analytical error estimates were derived for each method, and used to predict the number of subintervals required to satisfy various error tolerances. Numerical experiments across a variety of integrals confirmed the theoretical convergence rates and demonstrated that, in many cases, actual errors were significantly lower than predicted. Adaptive versions of the Midpoint and Trapezoidal rules were also implemented and, in certain cases, outperformed their non-adaptive counterparts in terms of computational efficiency.

2 Statement of the Problem

When approximating definite integrals numerically, the accuracy of a quadrature method often depends on the smoothness of the integrand and the method's degree of exactness. While basic quadrature rules can give simple and fast approximations, their accuracy can deteriorate over larger intervals or with complex integrands. Composite methods improve upon this by partitioning the interval into smaller subintervals and applying the basic rule locally, capturing finer features of the function and significantly reducing the overall error. This project explores the derivation, implementation, and performance of six such composite methods, assessing their theoretical error bounds and empirical behavior across a range of test integrals. The study also investigates how adaptive refinement strategies can further enhance efficiency by adjusting the subinterval size to meet a desired accuracy with fewer function evaluations.

3 Description of the Mathematics

Much of the mathematics utilized in this assignment can be found in Dr. Gallivan's class notes. Thus, we will not go into detail restating material from these notes. Instead, this section will satisfy tasks 1 and 4 of the assignment. We will derive the error expressions for the composite rules and then use these error expressions to estimate the subinterval size needed to satisfy various error demands.

3.1 Error Expression Derivations

The total error for the Composite Midpoint Rule is given in the slides as:

$$E_m^{cmp} = \frac{1}{24}(b-a)H_m^2 f^{(2)}(\eta) + \mathcal{O}(H_m^3).$$

The total error for the Composite Trapezoidal Rule is given in the slides as:

$$E_m^{ctr} = -\frac{1}{12}(b-a)H_m^2 f^{(2)}(\eta) + \mathcal{O}(H_m^3).$$

The total error for the Composite Simpson's First Rule is given in the slides as:

$$E_m^{csf} = -\frac{1}{2880}(b-a)H_m^4 f^{(4)}(\eta) + \mathcal{O}(H_m^5).$$

We will now satisfy Task 1 of the assignment, deriving the error expressions for the Composite Left Rectangle Rule, the Composite Open Newton-Cotes two-point Method, and the Composite Gauss-Legendre two-point Method.

Consider the Composite Left Rectangle Rule. The local error can be expressed as

$$E_0 = \mathcal{I}(f) - I_0(f) = \int_{a_i}^{b_i} f(x)dx - H_m f(a_i).$$

We can use a Taylor expansion to obtain

$$\begin{aligned}\int_{a_i}^{b_i} f(x)dx &= \int_{a_i}^{b_i} \left[f(a_i) + f'(a_i)(x-a_i) + \frac{f''(a_i)}{2}(x-a_i)^2 \right] dx \\ &= H_m f(a_i) + \frac{H_m}{2} f'(a_i) + \frac{H_m^3}{6} f''(a_i).\end{aligned}$$

Then,

$$\begin{aligned}E_0 = \mathcal{I}(f) - I_0(f) &= H_m f(a_i) + \frac{H_m}{2} f'(a_i) + \frac{H_m^3}{6} f''(a_i) - H_m f(a_i) \\ &= \frac{H_m}{2} f'(a_i) + \frac{H_m^3}{6} f''(a_i).\end{aligned}$$

So, the local error is

$$E_0 = \frac{1}{2} H_m^2 f'(\eta) + \mathcal{O}(H_m^3).$$

Thus, the total error is

$$\begin{aligned}E_m^{clr} &= m\left(\frac{1}{2} H_m^2 f'(\eta)\right) \\ &= m \cdot \frac{1}{2} \cdot \frac{(b-a)^2}{m^2} f'(\eta) \\ &= \frac{1}{2}(b-a)H_m f'(\eta).\end{aligned}$$

Now, consider the Composite Open Newton-Cotes two-point Method. By Theorem 24.5 in the class notes, the local error can be expressed as

$$\begin{aligned} E_1(f) &= \frac{\int_{-1}^{2}(t-0)(t-1)dt}{2}h^3 f^{(2)}(\eta) \\ &= \frac{1.5}{2}h^3 f^{(2)}(\eta) \\ &= \frac{3}{4}h^3 f^{(2)}(\eta). \end{aligned}$$

Note that $h = \frac{(b-a)}{3m}$. Thus, the total error is

$$\begin{aligned} E_m^{onc} &= m\left(\frac{3}{4}h^3 f^{(2)}(\eta)\right) \\ &= m \cdot \frac{3}{4} \cdot \frac{(b-a)^3}{27m^3} f^{(2)}(\eta) \\ &= \frac{1}{36}(b-a)H_m^2 f^{(2)}(\eta). \end{aligned}$$

Now, consider the Composite Gauss-Legendre two-point Method. By Theorem 25.3 in the class notes, there exists a $\xi \in (-1,1)$ such that the error when using Gauss-Legendre to approximate an integral on $[-1,1]$ is

$$E_1(f) = \frac{2^5\left\{(2)!\right\}^4}{(5)\left\{(4)!\right\}^3} f^{(4)}(\xi) = \frac{1}{135} f^{(4)}(\xi).$$

To apply this rule over a general subinterval $[x_i, x_{i+1}]$, we apply the change of variables

$$x = \frac{x_{i+1}+x_i}{2} + \frac{H_m}{2}\xi,$$

which maps $\xi \in [-1,1]$ to $x \in [x_i, x_{i+1}]$. Then $dx = \frac{H_m}{2}d\xi$ and

$$\int_{x_i}^{x_{i+1}} f(x)dx = \frac{H_m}{2}\int_{-1}^{1} f(\xi)d\xi.$$

Then, over our desired interval,

$$\begin{aligned} E_1(f) &= \frac{1}{135} \cdot \frac{H_m}{2} f^{(4)}(\xi) \\ &= \frac{1}{135} \cdot \frac{H_m}{2} \cdot f^{(4)}(x(\xi)) \cdot \left(\frac{dx}{d\xi}\right)^4 \\ &= \frac{1}{135}\left(\frac{H_m}{2}\right)^5 f^{(4)}(x(\xi)) \\ &= \frac{1}{135}\left(\frac{H_m}{2}\right)^5 f^{(4)}(\eta). \end{aligned}$$

Thus, the total error is

$$\begin{aligned} E_m^{2GL} &= m\left(\frac{1}{135}\left(\frac{H_m}{2}\right)^5 f^{(4)}(\eta)\right) \\ &= m\left(\frac{1}{135}\cdot\frac{(b-a)^5}{32m^5}f^{(4)}(\eta)\right) \\ &= \frac{1}{4320}(b-a)H_m^4 f^{(4)}(\eta). \end{aligned}$$

3.2 Optimal Subinterval Size Derivations

We will now satisfy Task 4 of the assignment, using the error expressions for the composite quadrature methods to estimate the subinterval size needed to satisfy various error demands. Suppose we have a desired error tolerance ϵ. This section will solve for the H_m that will keep the error expressions stated or derived in Section 3.1 less than ϵ. Note that in this section, $||\cdot||$ represents $||\cdot||_\infty$, or the maximum value of the function over the interval.

Composite Midpoint Rule:

$$\begin{aligned} \left|\frac{1}{24}(b-a)H_m^2 f^{(2)}(\eta)\right| &< \epsilon \\ \frac{(b-a)}{24}H_m^2||f^{(2)}(\eta)|| &< \epsilon \\ H_m &< \sqrt{\frac{24\epsilon}{(b-a)||f^{(2)}(\eta)||}} \end{aligned}$$

Composite Trapezoidal Rule:

$$\begin{aligned} \left|-\frac{1}{12}(b-a)H_m^2 f^{(2)}(\eta)\right| &< \epsilon \\ \frac{(b-a)}{12}H_m^2||f^{(2)}(\eta)|| &< \epsilon \\ H_m &< \sqrt{\frac{12\epsilon}{(b-a)||f^{(2)}(\eta)||}} \end{aligned}$$

Composite Simpson's First Rule:

$$\begin{aligned} \left|-\frac{1}{2880}(b-a)H_m^4 f^{(4)}(\eta)\right| &< \epsilon \\ \frac{(b-a)}{2880}H_m^4||f^{(4)}(\eta)|| &< \epsilon \\ H_m &< \sqrt[4]{\frac{2880\epsilon}{(b-a)||f^{(4)}(\eta)||}} \end{aligned}$$

Composite Left Rectangle Rule:

$$\left|\frac{1}{2}(b-a)H_m f'(\eta)\right| < \epsilon$$

$$\frac{(b-a)}{2}H_m||f'(\eta)|| < \epsilon$$

$$H_m < \frac{2\epsilon}{(b-a)||f'(\eta)||}$$

Composite Open Newton-Cotes two-point Method:

$$\left|\frac{1}{36}(b-a)H_m^2 f^{(2)}(\eta)\right| < \epsilon$$

$$\frac{(b-a)}{36}H_m^2||f^{(2)}(\eta)|| < \epsilon$$

$$H_m < \sqrt{\frac{36\epsilon}{(b-a)||f^{(2)}(\eta)||}}$$

Composite Gauss-Legendre two-point Method:

$$\left|\frac{1}{4320}(b-a)H_m^4 f^{(4)}(\eta)\right| < \epsilon$$

$$\frac{(b-a)}{4320}H_m^4||f^{(4)}(\eta)|| < \epsilon$$

$$H_m < \sqrt[4]{\frac{4320\epsilon}{(b-a)||f^{(4)}(\eta)||}}$$

4 Description of the Algorithm and Implementation

All routines and experiments were developed and run in Google Colab, a cloud-based platform provided by Google that allows users to write and execute Python code in a Jupyter Notebook environment.

Routines were developed for the Composite Midpoint Rule, Composite Trapezoidal Rule, Composite Simpson's First Rule, Composite Left Rectangle Rule, Composite Open Newton-Cotes two-point Method, and Composite Gauss-Legendre two-point Method. For the Composite Midpoint Rule and Composite Trapezoidal Rule, routines were also developed to implement an efficient adaptive global mesh refinement. This section will satisfy Task 3 of the assignment, discussing the efficient implementation of the routines

4.1 Basic Composite Quadrature Routines

The Composite Midpoint Rule approximates the integral by evaluating the function at the midpoint of each of the m subintervals and summing the contributions. The implementation first computes all midpoints using np.linspace between $a + \frac{H_m}{2}$ and $b - \frac{H_m}{2}$. The function is then evaluated exactly m times, once per subinterval, which is minimal for this rule. The approximation is calculated by multiplying the sum of these m function evaluations with the subinterval size H_m. Storage and operational complexity is low: only the array of midpoints is needed, and the method avoids redundant computations.

The Composite Trapezoidal Rule approximates the integral by linearly interpolating the function across each subinterval and averaging the values at endpoints. The implementation uses np.linspace to generate $m + 1$ evenly spaced nodes and then computes the appropriate weighted sum. Only $m + 1$ evaluations are required, with endpoints used once and interior points weighted by 2, making the method highly efficient in time. Storage requirements are also low, as only the array on nodes is stored.

Composite Simpson's First Rule integrates over each subinterval using a quadratic polynomial determined by values at the endpoints and the midpoint. This implementation calculates the mesh of $m + 1$ endpoints using np.linspace and generates midpoint values by adding $\frac{H_m}{2}$ to the left endpoint of each subinterval. The rule combines the function values with weights of 1 (global endpoints), 2 (interior endpoints), and 4 (midpoints), yielding an efficient and accurate approximation. The function is evaluated $2m + 1$ times, which is more than the midpoint or trapezoidal rule but justified by the improved accuracy. No additional storage beyond the mesh and midpoints is required.

The Composite Left Rectangle Rule approximates the integral using the value of the function at the left endpoint of each subinterval. The code uses np.linspace to generate these left endpoints and computes the sum of their function values, scaled by the subinterval width. It requires exactly m function evaluations, one per subinterval, and is efficient in both time and space.

The Composite Open Newton-Cotes two-point Method evaluates the function at the $\frac{1}{3}$ and $\frac{2}{3}$ points of each subinterval and applies an appropriate weighted sum. The implementation loops over subintervals, computing local nodes and applying the rule with a total of $2m$ function evaluations. The code avoids unnecessary storage and handles all computations locally within the loop, making it memory-efficient.

The Composite Gauss-Legendre two-point Method approximates the integral Gauss-Legendre quadrature, which exactly integrates polynomials up to degree 3. Since the weights for the two-point rule are both 1, the implementation avoids storing or referencing a weights array, simplifying the computation. For each of the m subintervals, the code transforms the fixed nodes $\pm\frac{1}{\sqrt{3}}$ to the local subinterval and computes the weighted average directly as a simple sum of function evaluations. The method requires $2m$ function evaluations in total and does not store any intermediate values beyond the transformed nodes, making it very memory efficient. The reuse of fixed nodes and implicit weights leads to reduced storage requirements.

4.2 Adaptive Global Mesh Refinement Routines

The Adaptive Composite Trapezoidal Method implements adaptive refinement of the composite trapezoidal rule by successively halving the subinterval width H and doubling the number of subintervals m until the desired accuracy is achieved or the maximum number of iterations is reached. It is an efficient algorithm, as it reuses previously computed function evaluations. At each refinement step, only the new midpoints between existing nodes are evaluated. This is done efficiently by iterating through the midpoint locations and summing their contributions. The integral is updated using the formula $I_{\text{new}} = \frac{1}{2} I_{\text{old}} + H \sum f(x_{\text{new midpoints}})$, which avoids recomputation of existing values. The method is efficient in both time and space: it stores only scalars and reuses the previous integral estimate. This reuse drastically reduces the number of function calls required.

The Adaptive Composite Midpoint routine refines the mesh by splitting each subinterval into three equal parts at each step (i.e., $H \to \frac{H}{3}, m \to 3m$), as required for efficient reuse based on the midpoint rule's open form. Function values are only computed at newly introduced midpoints, specifically those at $\frac{H}{2}$ and $\frac{5H}{2}$ relative to each original subinterval, allowing the code to avoid redundant evaluations. The new integral is computed via the recurrence $I_{\text{new}} = \frac{1}{3} I_{\text{old}} + H \sum f(x_{\text{new}})$, which leverages prior estimates. This structured reuse significantly improves efficiency, as the number of function evaluations grows more slowly than it would without reuse. As with the Adaptive Composite Trapezoidal Method, storage complexity is low since the routine reuses the previous integral estimate.

5 Description of the Experimental Design and Results

5.1 Routine Validation

In this section, we will satisfy Task 2 of the assignment, empirically validating the correct functioning of the routines. To do so, we will use four test functions:

1. $f(x) = e^x$ on $[0, 3]$. Chosen because it is a smooth function with a significant change in variation.

2. $f(x) = \sin(x)$ on $[0, 1]$. Chosen because of its smooth, bounded, and oscillatory behavior.

3. $f(x) = x^3 + 2x^2 - x + 5$ on $[0, 1]$. Chosen because it is a cubic polynomial and can be used to test degree of exactness behavior.

4. $f(x) = \frac{1}{\sqrt{x}}$ on $[0.01, 1]$. Chosen because of its steep gradient near $x = 0$.

Figure 1 shows the results of approximating the definite integrals of the four given functions (and bounds) using all six composite quadrature methods with a varying number of subintervals. As expected, each method displays a convergence rate consistent with its theoretical order. For instance, the Left Rectangle Rule, which is first-order, exhibits linear decay on the log-log scale, while Simpson's First Rule and Gauss–Legendre quadrature, both of which have fourth-order accuracy, show much steeper slopes, indicating faster convergence. For the cubic polynomial $f(x) = x^3 + 2x^2 - x + 5$, the error curves for Simpson's Rule and

Gauss–Legendre flatten near machine precision, consistent with the fact that these methods integrate cubic functions exactly. This behavior confirms both the degree of exactness and the correct implementation of the higher-order methods. The function $f(x) = \frac{1}{\sqrt{x}}$, which has a steep gradient near the left endpoint, shows a more gradual error decay across all methods. This slower convergence is expected due to the large magnitude of the second and higher derivatives near $x = 0.01$, and it highlights the impact of function regularity on quadrature accuracy. Nonetheless, the observed convergence trends still align with theoretical expectations, further verifying correctness. The results for $f(x) = \sin(x)$ demonstrate smooth convergence for all methods, with again the relative performance matching theoretical predictions. Importantly, across all four test cases, the error decreases monotonically with increasing m, and the relative performance ordering of the methods is consistent, suggesting correct implementation.

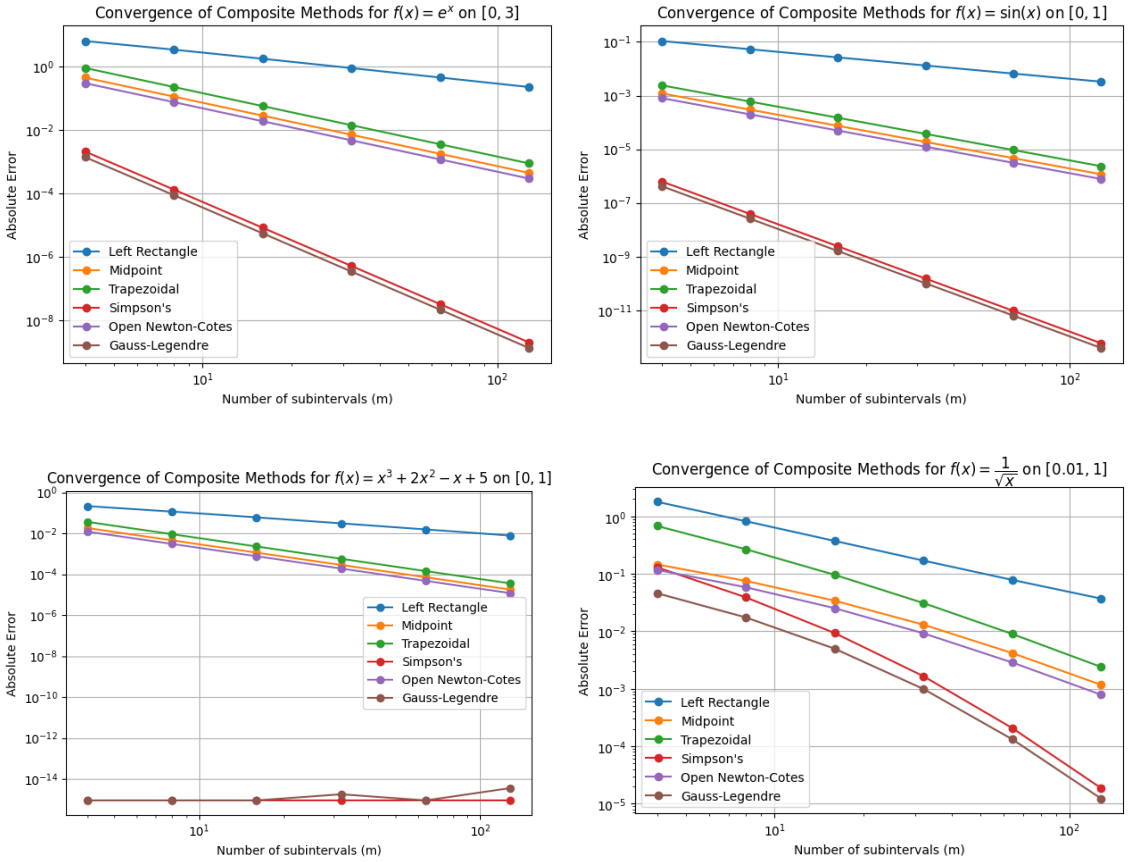

Figure 1. Convergence behavior of the six composite quadrature methods for four well-balanced functions. Error was computed for varying numbers of subintervals ($m = 4, 8, 16, 32, 64, 128$). A log-log scale plot is utilized to better show behavior.

Figure 2 shows the theoretical error bounds for each of the six composite quadrature methods alongside their corresponding observed errors for the function $f(x) = e^x$ on $[0, 3]$. Theoretical bounds were computed using the composite error expressions derived in Section 3.1. These depend on the global interval length, subinterval size H_m, and appropriate derivatives of f. The bounds themselves are conservative, and the fact that no observed error violates its bound provides strong empirical validation of both the analytical error estimates and

the correctness of the numerical implementations. Moreover, the log-log scale emphasizes that the slopes of the error curves match the theoretical predictions, further supporting the correctness of each quadrature method.

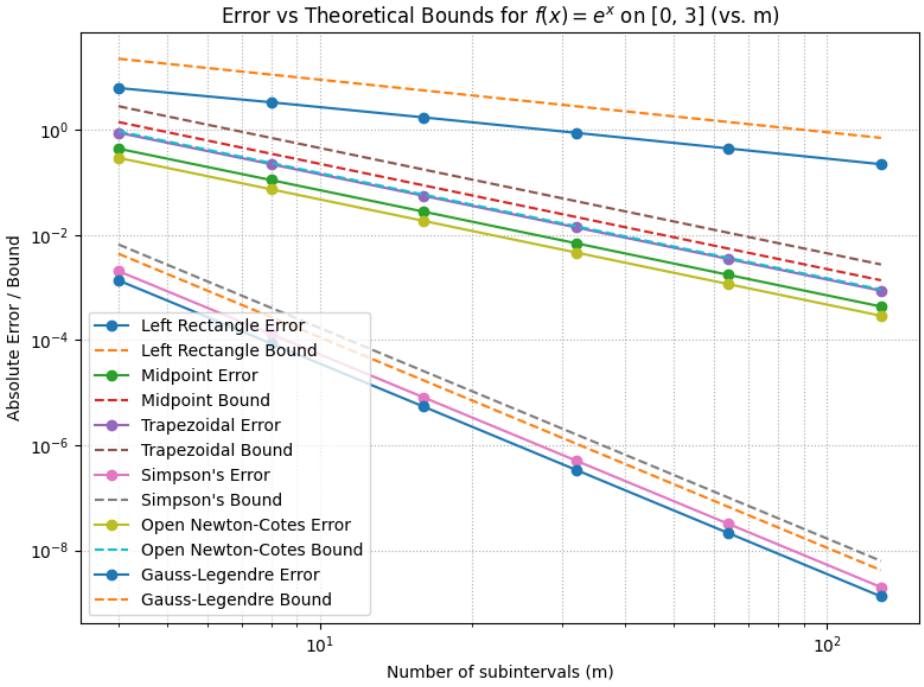

Figure 2. Error plotted with the theoretical error bound for each of the six composite quadrature methods for $f(x) = e^x$ on $[0, 3]$. Error was computed for varying numbers of subintervals ($m = 4, 8, 16, 32, 64, 128$). A log-log scale plot is utilized to better show behavior.

5.2 Subinterval Sizes to Satisfy Error Requirements

In this section, we will demonstrate the number of subintervals and, consequently, the subinterval size, needed to satisfy various error requirements when approximating some definite integrals. The integrals tested are:

1. $\int_0^3 e^x \, dx$
2. $\int_0^{\frac{\pi}{3}} e^{\sin(2x)} \cos(2x) \, dx$
3. $\int_{-2}^1 \tanh(x) \, dx$
4. $\int_0^{3.5} x \cos(2\pi x) \, dx$
5. $\int_{0.1}^{2.5} \left(x + \frac{1}{x}\right) dx$

To get a sense for the behavior, we will show the results of testing each of the six composite methods with a different two definite integrals. For all tests except for composite left rectangle (for reasons that will be discussed later), four varying error tolerances are tested:

$\epsilon = 10^{-3}, 10^{-6}, 10^{-9}, \text{and } 10^{-12}$. Tables 1 and 2 show the results of using the Composite Midpoint Rule to approximate integrals 1 and 4. It can clearly be seen that as the error tolerance decreases, more and more subintervals are needed. At each error tolerance, significantly more subintervals are needed to approximate integral 4. This makes sense due to the highly oscillatory nature of this function and its derivatives. Interestingly, we get much more accuracy than expected when approximating integral 4. For each error tolerance, the actual error observed when using the optimal number of subintervals is at least 2 orders of magnitude better than we would expect.

ϵ	10^{-3}	10^{-6}	10^{-9}	10^{-12}
Optimal m	151	4754	150321	4753550
Optimal H_m	1.98675e-02	6.31048e-04	1.99573e-05	6.31107e-07
Error	3.13889e-04	3.16678e-07	3.16732e-10	3.19744e-13

Table 1: Optimal Subinterval Sizes for approximating $\int_0^3 e^x dx$ using the Composite Midpoint Rule and various error tolerances.

ϵ	10^{-3}	10^{-6}	10^{-9}	10^{-12}
Optimal m	497	15712	496833	15711236
Optimal H_m	7.04225e-03	2.22760e-04	7.04462e-06	2.22771e-07
Error	4.13349e-06	4.13516e-09	4.13655e-12	4.38538e-15

Table 2: Optimal Subinterval Sizes for approximating $\int_0^{3.5} x\cos(2\pi x)dx$ using the Composite Midpoint Rule and various error tolerances.

Tables 3 and 4 show the results of using the Composite Trapezoidal Rule to approximate integrals 2 and 5. We see a similar obvious trend that the number of required subintervals increases as the error tolerance decreases. It takes many more subintervals to accurately approximate integral 5 for each error tolerance. This is likely due to the high maximum value of the second derivative, which pushes down the optimal subinterval size. Again, with the high number of subintervals needed to satisfy the error tolerance, we see an entire order of magnitude more accuracy than we would expect.

ϵ	10^{-3}	10^{-6}	10^{-9}	10^{-12}
Optimal m	40	1249	39475	1248305
Optimal H_m	2.61799e-02	8.38429e-04	2.65281e-05	8.38896e-07
Error	2.81558e-04	2.88750e-07	2.89069e-10	2.88547e-13

Table 3: Optimal Subinterval Sizes for approximating $\int_0^{\pi/3} e^{\sin(2x)}\cos(2x)dx$ using the Composite Trapezoidal Rule and various error tolerances.

ϵ	10^{-3}	10^{-6}	10^{-9}	10^{-12}
Optimal m	1518	48000	1517894	48000000
Optimal H_m	1.58103e-03	5.00000e-05	1.58114e-06	5.00000e-08
Error	2.07966e-05	2.08000e-08	2.08011e-11	3.46390e-14

Table 4: Optimal Subinterval Sizes for approximating $\int_{0.1}^{2.5} x + \frac{1}{x} dx$ using the Composite Trapezoidal Rule and various error tolerances.

Tables 5 and 6 show the results of using the Composite Simpson's First Rule to approximate integrals 3 and 5. Here, we see better accuracy than expected for both functions. Approximating integral 3 does not take a large amount of subintervals for each tolerance, which makes sense due to the smooth and well-behaved nature of the hyperbolic tangent function. It takes more subintervals to approximate integral 5, but nowhere near as many as it took the trapezoidal rule to approximate the same function. This highlights the remarkable accuracy of Simpson's First quadrature. We see significantly better error than expected here, with error being at least 2 orders of magnitude less than the tolerance.

ϵ	10^{-3}	10^{-6}	10^{-9}	10^{-12}
Optimal m	5	25	137	767
Optimal H_m	6.00000e-01	1.20000e-01	2.18978e-02	3.91134e-03
Error	1.98280e-05	2.67554e-08	2.94653e-11	3.03091e-14

Table 5: Optimal Subinterval Sizes for approximating $\int_{-2}^{1} \tanh(x) dx$ using the Composite Simpson's First Rule and various error tolerances.

ϵ	10^{-3}	10^{-6}	10^{-9}	10^{-12}
Optimal m	91	508	2855	16050
Optimal H_m	2.63736e-02	4.72441e-03	8.40630e-04	1.49533e-04
Error	9.69068e-06	1.03651e-08	1.04032e-11	9.76996e-15

Table 6: Optimal Subinterval Sizes for approximating $\int_{0.1}^{2.5} x + \frac{1}{x} dx$ using the Composite Simpson's First Rule and various error tolerances.

Tables 7 and 8 show the results of using the Composite Left Rectangle Rule to approximate integrals 1 and 3. Due to the very poor performance of the left rectangle rule, relaxed error tolerances were used for these tests. The code would consistently crash when trying to use stricter tolerances. Very high numbers of subintervals are needed for both integrals, however the situation is worse for integral 1. This is due to the exponential nature of the function. The left rectangle rule is providing a very bad approximation here because of the dramatic increase in function values. Hence, many many subintervals are needed even for mild error tolerances. In neither case do we see greater accuracy than expected, adding to the issues seen with this method.

ϵ	10^{-3}	10^{-4}	10^{-5}	10^{-6}
Optimal m	90385	903850	9038492	90384917
Optimal H_m	3.31913e-05	3.31913e-06	3.31914e-07	3.31914e-08
Error	3.16736e-04	3.16737e-05	3.16738e-06	3.16738e-07

Table 7: Optimal Subinterval Sizes for approximating $\int_0^3 e^x dx$ using the Composite Left Rectangle Rule and various error tolerances.

ϵ	10^{-3}	10^{-4}	10^{-5}	10^{-6}
Optimal m	4500	45000	450000	4500000
Optimal H_m	6.66667e-04	6.66667e-05	6.66667e-06	6.66667e-07
Error	5.75194e-04	5.75206e-05	5.75207e-06	5.75207e-07

Table 8: Optimal Subinterval Sizes for approximating $\int_{-2}^1 \tanh(x)dx$ using the Composite Left Rectangle Rule and various error tolerances.

Tables 9 and 10 show the results of using the Composite Open Newton-Cotes two-point Method to approximate integrals 2 and 4. In these tests, we see an exponential increase in the number of subintervals needed as error tolerance gets stricter. This phenomenon is more pronounced with integral 4 due to its highly oscillatory nature, but here we do see more accuracy than expected with each error tolerance.

ϵ	10^{-3}	10^{-6}	10^{-9}	10^{-12}
Optimal m	23	721	22791	720710
Optimal H_m	4.55303e-02	1.45242e-03	4.59479e-05	1.45301e-06
Error	2.83954e-04	2.88838e-07	2.89059e-10	2.97873e-13

Table 9: Optimal Subinterval Sizes for approximating $\int_0^{\pi/3} e^{\sin(2x)}\cos(2x)dx$ using the Composite 2-point Open Newton-Cotes Method and various error tolerances.

ϵ	10^{-3}	10^{-6}	10^{-9}	10^{-12}
Optimal m	406	12829	405663	12828170
Optimal H_m	8.62069e-03	2.72819e-04	8.62785e-06	2.72837e-07
Error	4.12956e-06	4.13502e-09	4.13296e-12	2.96915e-14

Table 10: Optimal Subinterval Sizes for approximating $\int_0^{3.5} x\cos(2\pi x)dx$ using the Composite 2-point Open Newton-Cotes Method and various error tolerances.

Tables 11 and 12 show the results of using the Composite Gauss-Legendre two-point Method to approximate integrals 1 and 5. It takes a low number of subintervals to achieve each error tolerance when approximating integral 1 and a slightly higher, but still relatively low number to accurately approximate integral 5 to each tolerance. This emphasizes the high order of Gauss-Legendre quadrature. It likely takes more subintervals to approximate integral 5 due to the very high maximum fourth derivative on the interval that causes the optimal subinterval size to diminish. However, when approximating this integral, we do see much better accuracy than expected for each error tolerance.

ϵ	10^{-3}	10^{-6}	10^{-9}	10^{-12}
Optimal m	6	33	184	1031
Optimal H_m	5.00000e-01	9.09091e-02	1.63043e-02	2.90980e-03
Error	2.73945e-04	3.01673e-07	3.12198e-10	3.41061e-13

Table 11: Optimal Subinterval Sizes for approximating $\int_0^3 e^x dx$ using the Composite 2-point Gauss-Legendre Method and various error tolerances.

ϵ	10^{-3}	10^{-6}	10^{-9}	10^{-12}
Optimal m	82	459	2579	14503
Optimal H_m	2.92683e-02	5.22876e-03	9.30593e-04	1.65483e-04
Error	9.68372e-06	1.03635e-08	1.04210e-11	3.01981e-14

Table 12: Optimal Subinterval Sizes for approximating $\int_{0.1}^{2.5} x + \frac{1}{x}dx$ using the Composite 2-point Gauss-Legendre Method and various error tolerances.

The observed results confirm theoretical predictions: smoother integrands and higher-order methods generally require fewer subintervals to achieve a given accuracy. Notably, Simpson's First Rule and Gauss-Legendre quadrature achieved error levels significantly below the prescribed tolerance. There also seems to be many cases for varying methods where a large number of subintervals are required and yet we get greater accuracy than expected. In contrast, lower-order methods like the Left Rectangle Rule required an impractically large number of subintervals and failed to deliver better-than-expected accuracy, highlighting their inefficiency for high-accuracy integration.

5.3 Adaptive Composite Quadrature Comparisons

In this section, we will demonstrate the number of subintervals needed to satisfy various error requirements when approximating definite integrals using the Adaptive Composite Midpoint

Method and the Adaptive Composite Trapezoidal Method. For the sake of comparison, we will test both methods with the same integrals that we tested their non-adaptive counterparts with in Section 5.2. For all tests, we use three varying error tolerances: $\epsilon = 10^{-3}, 10^{-6}$, and 10^{-9}. We will not test with 10^{-12} as we did in the last section as the adaptive code could not handle these strict error tolerances in most cases. Tables 13 and 14 show the results of using the Adaptive Composite Midpoint Method to approximate integrals 1 and 4. As in the non-adaptive case, we need more subintervals for stricter error tolerances. Interestingly, when compared to the non-adaptive tests, this method is worse for integral 1 and better for integral 4. When approximating integral 1, we need more subintervals to reach each error tolerance than we did in the non-adaptive case. When approximating integral 4, we need significantly fewer subintervals to satisfy each error tolerance than we did in the non-adaptive case. However, we do not see the trend of much greater accuracy than expected.

ϵ	10^{-3}	10^{-6}	10^{-9}
Optimal m	243	6561	177147
Error	1.21205e-04	1.66264e-07	2.24830e-10

Table 13: Number of Subintervals and Errors for approximating $\int_0^3 e^x dx$ using the Adaptive Composite Midpoint Rule and various error tolerances.

ϵ	10^{-3}	10^{-6}	10^{-9}
Optimal m	81	2187	59049
Error	1.56598e-04	2.13433e-07	2.89080e-10

Table 14: Number of Subintervals and Errors for approximating $\int_0^{3.5} x\cos(2\pi x)dx$ using the Adaptive Composite Midpoint Rule and various error tolerances.

Tables 15 and 16 show the results of using the Adaptive Composite Trapezoidal Method to approximate integrals 2 and 5. Again, as in the non-adaptive case, we need more subintervals for stricter error tolerances. When compared to the non-adaptive tests, this method performs very similarly for integral 2, but much better for integral 5. When approximating integral 2, we see there is no huge difference in the number of subintervals needed to satisfy the various error requirements, while the absolute error remains about the same as well. When approximating integral 5, we need significantly fewer subintervals to satisfy each error tolerance than we did in the non-adaptive case. However, we again do not see any greater accuracy than expected.

ϵ	10^{-3}	10^{-6}	10^{-9}
Optimal m	32	1024	32768
Error	4.39957e-04	4.29582e-07	4.19594e-10

Table 15: Number of Subintervals and Errors for approximating $\int_0^{\pi/3} e^{\sin(2x)}\cos(2x)dx$ using the Adaptive Composite Trapezoidal Rule and various error tolerances.

ϵ	10^{-3}	10^{-6}	10^{-9}
Optimal m	256	8192	262144
Error	7.30609e-04	7.14111e-07	6.96110e-10

Table 16: Number of Subintervals and Errors for approximating $\int_{0.1}^{2.5} x + \frac{1}{x} dx$ using the Adaptive Composite Trapezoidal Rule and various error tolerances.

It appears that both the adaptive and non-adaptive versions of these methods can prove to be useful. Perhaps it is better to use the non-adaptive composite methods to approximate integrals and switch to adaptive methods if the number of subintervals needed to satisfy the desired error tolerance becomes unruly.

6 Conclusions

This project presented a detailed investigation of six composite quadrature methods and selected adaptive variants, emphasizing both theoretical predictions and empirical behavior. The routines were successfully implemented and validated using integrals with known exact solutions, consistently confirming the expected convergence rates and derived error bounds.

Across all methods, the number of subintervals needed to meet a given error tolerance followed theoretical trends - fewer for smoother integrands and more for functions with steep gradients or oscillations. Higher-order methods, such as Simpson's First Rule and Gauss-Legendre quadrature, achieved target accuracies with far fewer subintervals than lower-order methods. Observed errors were often one to two orders of magnitude below predicted bounds, indicating conservative estimates and well-implemented routines. In contrast, the Left Rectangle Rule consistently performed poorly, requiring excessive subintervals even under relaxed tolerances. Its failure to approximate functions like $f(x) = e^x$ under tighter constraints highlighted its practical limitations.

Adaptive refinement methods provided efficiency gains in certain cases. Though not always more accurate than non-adaptive routines, they often required fewer function evaluations, particularly for integrals with uneven function behavior. For example, in approximating $\int_0^{3.5} x \cos(2\pi x)\, dx$, the adaptive midpoint method required significantly fewer subintervals than its non-adaptive counterpart. However, adaptive methods did not consistently exhibit the enhanced accuracy sometimes seen in fixed-mesh results, suggesting their strength lies in reducing computational cost, rather than boosting precision.

Overall, the results validate the correctness of the implementations and the underlying theory. Discrepancies were explained by function characteristics or method limitations. The findings affirm that higher-order composite methods are highly effective for numerical integration, particularly when combined with adaptive refinement to optimize efficiency for complex integrands.

7 References

In the creation of this report, I referenced Dr. Gallivan's class notes. I also consulted with Brayton Link and Dan Mazus regarding algorithm development.